



HCM-UTE

TRƯỜNG ĐẠI HỌC

CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH

HCMC University of Technology and Engineering

HỌC PHẦN: TƯƠNG TÁC DỮ LIỆU TRỰC QUAN

Mã học phần: IDVI333677

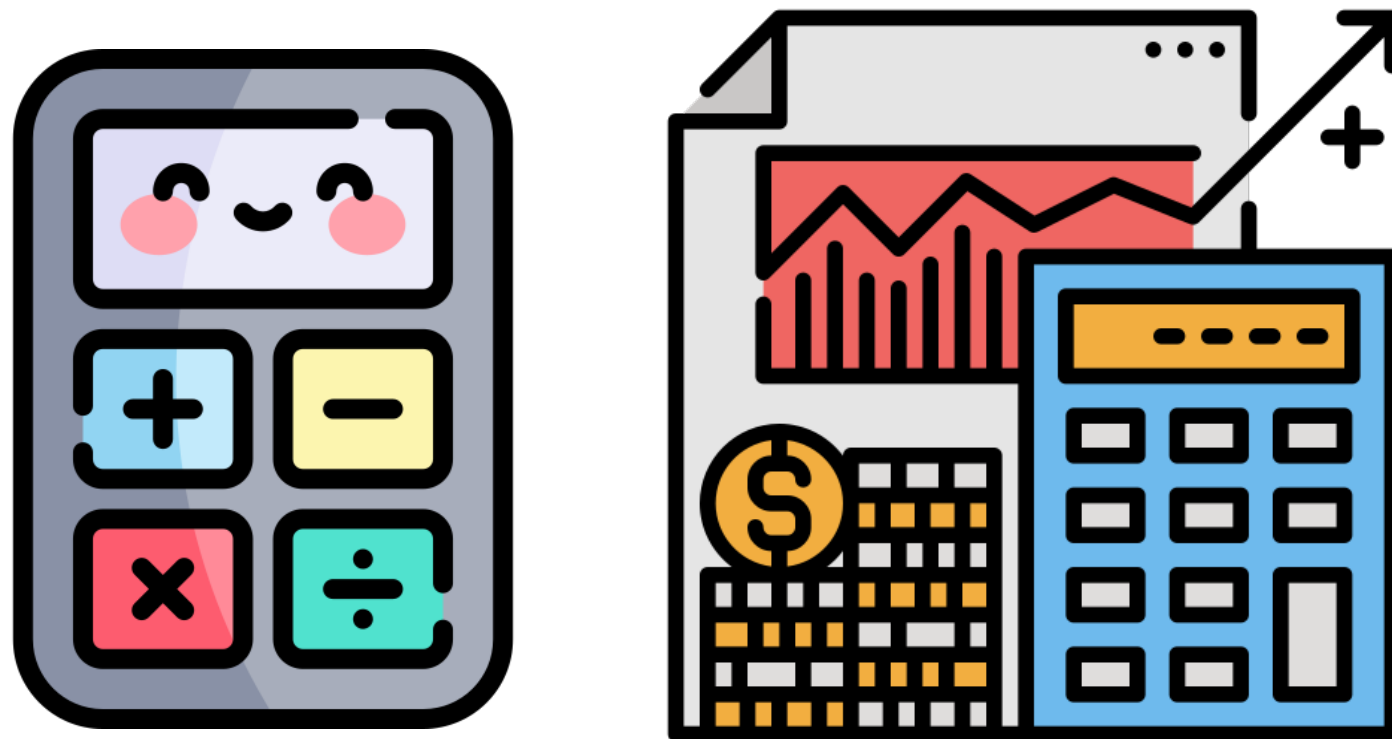
Giảng viên: Ths. Đoàn Minh Trí

Email liên hệ: doanminhtri@hcmute.edu.vn





HCM-UTE



CHƯƠNG 5: CÁC TRƯỜNG DỮ LIỆU TÍNH TOÁN

(Giảng dạy tích hợp nội dung chương 8)



MỤC TIÊU

1. **Hiểu bản chất:** Nắm vững vai trò của các trường dữ liệu tính toán (Calculated Fields) trong việc mở rộng và phân tích chuyên sâu tập dữ liệu thô.
2. **Thành thạo kỹ thuật:** Làm chủ các hàm tính toán tổng hợp, đếm khác biệt, khẩu phần/tỷ lệ, xử lý chuỗi và thời gian trong Python (pandas).
3. **Làm chủ kỹ thuật nâng cao:** Hiểu và áp dụng thành thạo khái niệm Mức chi tiết cố định (Fixed Level of Detail - Fixed LoD).
4. **Ứng dụng trực quan hóa:** Tích hợp dữ liệu tính toán vào các báo cáo và biểu đồ trực quan hóa đa chiều.

NỘI DUNG CHÍNH

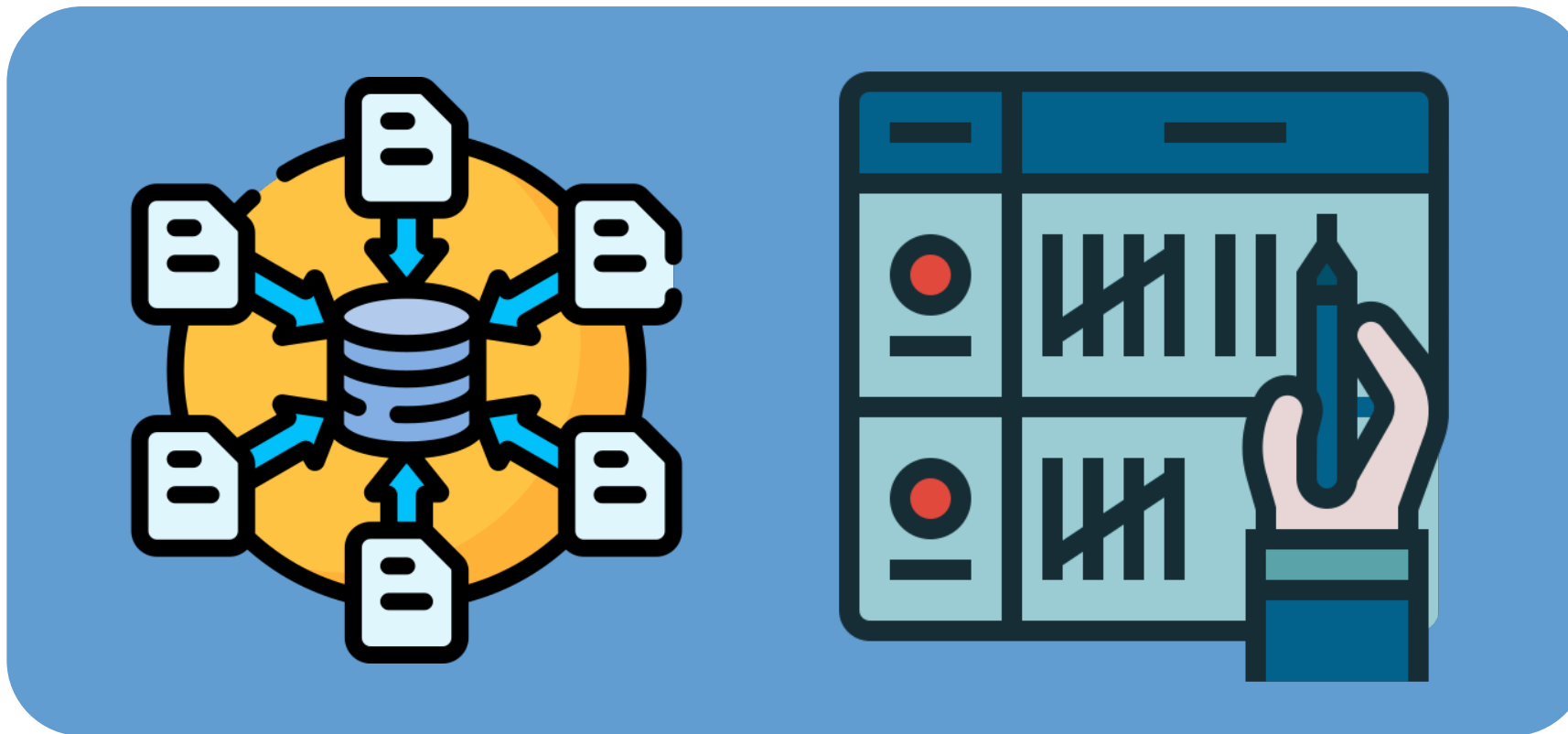
Phần 1: Tính tổng hợp cơ bản & Đếm khác biệt (Aggregations & Count Distinct)

Phần 2: Tính toán khẩu phần / Tỷ lệ (Ratio Calculations)

Phần 3: Xử lý Ngày tháng & Chuỗi ký tự (Date & String Functions)

Phần 4: Kỹ thuật Mức chi tiết cố định (Fixed Level of Detail - Fixed LoD)

Phần 5: Lập báo cáo & Trực quan hóa dữ liệu tính toán (Reporting & Visual Analytics)



PHẦN 1: TÍNH TỔNG HỢP CƠ BẢN & ĐẾM KHÁC BIỆT (AGGREGATIONS & COUNT DISTINCT)



1.1 - LÝ THUYẾT VỀ TÍNH TỔNG HỢP VÀ ĐẾM KHÁC BIỆT

Bản chất của Tính tổng hợp (Aggregations)

Gom nhóm dữ liệu: Chuyển đổi từ dữ liệu dòng chi tiết (*Row level*) lên mức độ tổng quát (*Aggregated level*).

Các chỉ số tổng hợp cơ bản:

- SUM (Tổng số),
- MEAN (Trung bình),
- MEDIAN (Trung vị),
- MIN/MAX.

1.1 - LÝ THUYẾT VỀ TÍNH TỔNG HỢP VÀ ĐẾM KHÁC BIỆT

Count vs. Count Distinct

Index	Normal Count and Distinct	Normal Count
1	Apple	???
2	Orange	
3	Pear	
4	Apple	
5	Apple	
6	Orange	
7	Apple	
8	Apple	???
9	Apple	
10	Apple	
11	Orange	
12	Pear	
13	Apple	
14	Apple	
15	Orange	



1.1 - LÝ THUYẾT VỀ TÍNH TỔNG HỢP VÀ ĐẾM KHÁC BIỆT

Tiêu chí	COUNT (Đếm)	COUNT DISTINCT (Đếm khác biệt)
Cách tính	Đếm tất cả các dòng dữ liệu không bị rỗng (null/NaN).	Đếm số lượng giá trị duy nhất trong tập dữ liệu.
Ví dụ minh họa	100 đơn hàng → kết quả là 100.	100 đơn hàng nhưng chỉ đến từ 15 khách hàng → kết quả là 15.
Ý nghĩa sử dụng	Cho biết tổng số bản ghi hoặc giao dịch.	Cho biết số lượng thực thể duy nhất (khách hàng, sản phẩm, ...).
Ưu điểm	Nhanh chóng, dễ hiểu, phản ánh khối lượng dữ liệu.	Tránh phóng đại số lượng thực thể khi một khách hàng/sản phẩm xuất hiện nhiều lần.
Nhược điểm	Có thể gây hiểu nhầm nếu một thực thể xuất hiện nhiều lần.	Tốn tài nguyên tính toán hơn, đặc biệt với dữ liệu lớn.

COUNT phù hợp khi muốn biết tổng số bản ghi.

COUNT DISTINCT hữu ích khi muốn biết số lượng thực thể duy nhất, tránh trùng lặp.



1.2 TÍNH TỔNG HỢP & NUNIQUE

```
import pandas as pd
import numpy as np

# Giả lập dữ liệu đơn hàng
np.random.seed(42)
df_orders = pd.DataFrame({
    'Order_ID': [f'ORD-{i:03d}' for i in range(1, 101)],
    'Customer_ID': np.random.choice([f'CUST-{i:02d}' for i in range(1, 21)], 100),
    'Category': np.random.choice(['Electronics', 'Clothing', 'Furniture'], 100),
    'Sales': np.random.randint(50, 500, 100)
})

# Tính Aggregate & Count Distinct theo Category
agg_summary = df_orders.groupby('Category').agg(
    Total_Sales=('Sales', 'sum'),           # Tính Tổng sales
    Avg_Sales=('Sales', 'mean'),           # Tính Sales trung bình
    Total_Orders=('Order_ID', 'count'),     # Đếm tổng số đơn hàng
    Unique_Customers=('Customer_ID', 'nunique') # Đếm khác biệt khách hàng (Count Distinct)
).reset_index()
print(agg_summary)
```

	Category	Total_Sales	Avg_Sales	Total_Orders	Unique_Customers
0	Clothing	7831	279.678571	28	16
1	Electronics	10392	288.666667	36	17
2	Furniture	10552	293.111111	36	17

1.3 - TRỰC QUAN HÓA KẾT QUẢ AGGREGATIONS

*Kết hợp với code
tại mục 1.2...*

```
import matplotlib.pyplot as plt

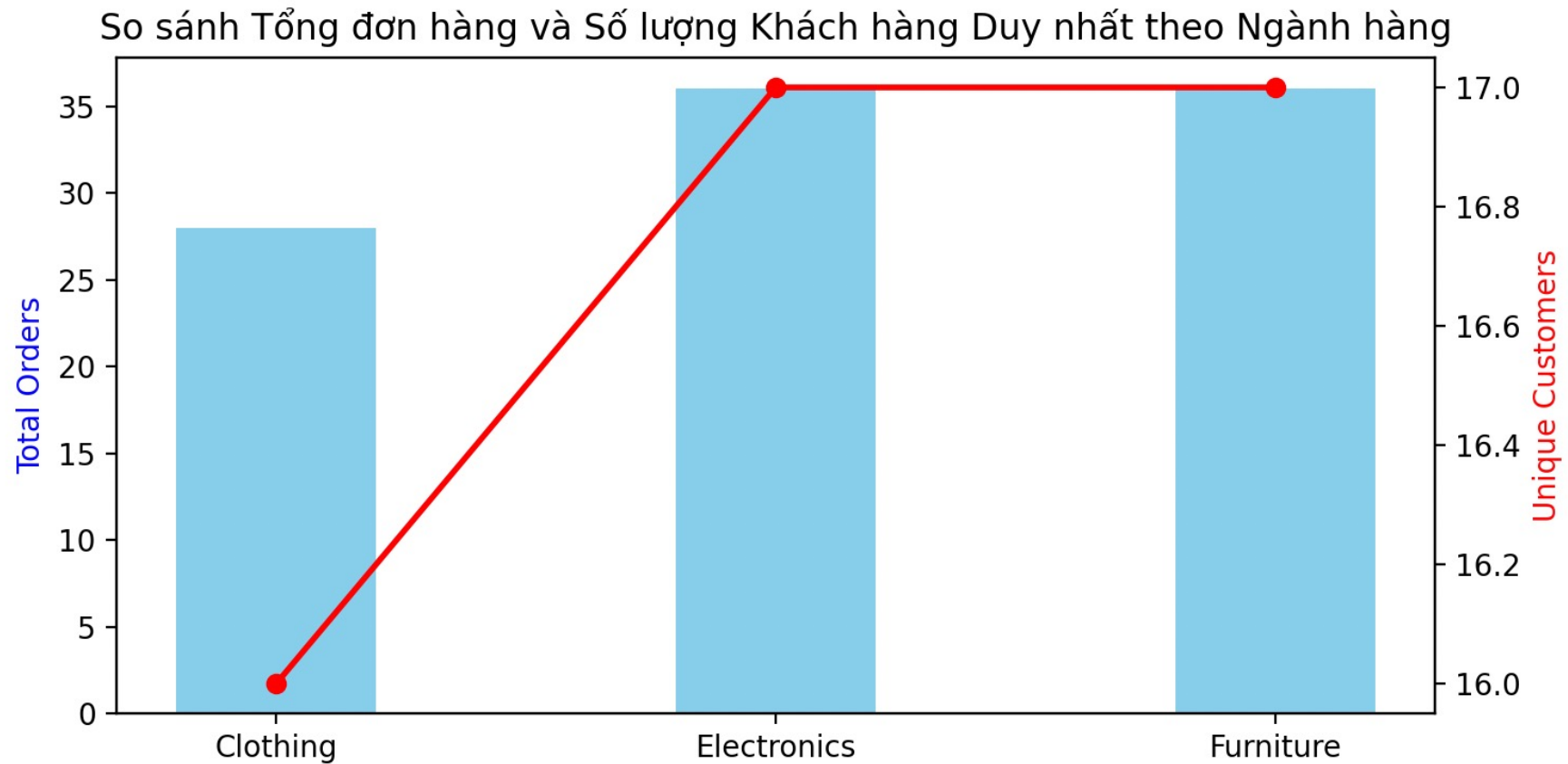
fig, ax1 = plt.subplots(figsize=(8, 4))

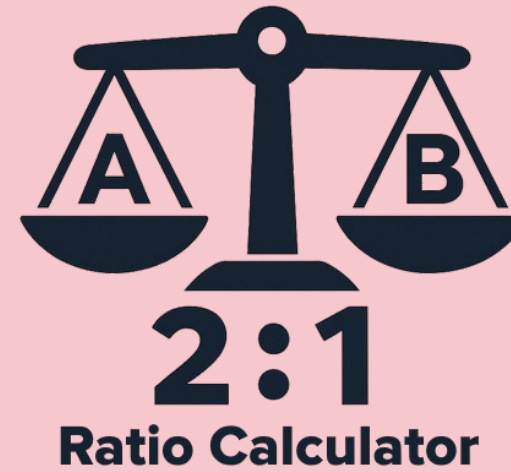
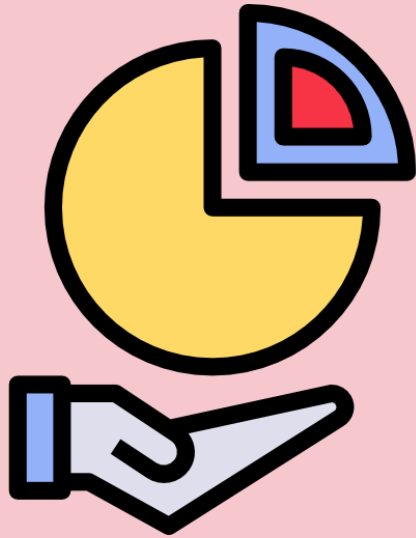
# Vẽ biểu đồ cột cho Tổng số đơn hàng (Count)
ax1.bar(agg_summary['Category'], agg_summary['Total_Orders'],
        color='skyblue', label='Total Orders', width=0.4)
ax1.set_ylabel('Total Orders', color='blue')

# Tạo trục Y thứ 2 để vẽ đường biểu diễn Số khách duy nhất (Count Distinct)
ax2 = ax1.twinx()
ax2.plot(agg_summary['Category'], agg_summary['Unique_Customers'],
         color='red', marker='o', linewidth=2, label='Unique Customers')
ax2.set_ylabel('Unique Customers', color='red')

plt.title('So sánh Tổng đơn hàng và Số lượng Khách hàng Duy nhất theo Ngành hàng')
plt.show()
```

1.3 - TRỰC QUAN HÓA KẾT QUẢ AGGREGATIONS





PHẦN 2: TÍNH TOÁN KHẤU PHẦN / TỶ LỆ (RATIO CALCULATIONS)



2.1 - BẢN CHẤT CỦA TÍNH TỶ LỆ TRONG PHÂN TÍCH

Các dạng bài toán Tỷ lệ (Ratio Calculations)

Dạng 1: Tỷ lệ đóng góp (Percentage of Total): Phản ánh thị phần hoặc tỷ trọng của từng danh mục trên tổng thể dữ liệu toàn cục.

$$\text{Percent of Total} = \frac{\text{Sum}(\text{Sales}_{\text{Group}})}{\text{Sum}(\text{Sales}_{\text{Overall}})} \times 100$$

Dạng 2: Tỷ lệ đóng góp nội bộ (Percentage within Group): Tỷ lệ của sub-category nằm trong nhóm mẹ.

Dạng 3: Chỉ số hiệu suất / Khẩu phần (Ratio Metrics): Lập tỷ lệ giữa 2 biến số tính toán khác nhau vd:

$$\text{Profit Margin} = \frac{\text{Profit}}{\text{Sales}}$$



2.2 - HƯỚNG DẪN PYTHON: TÍNH TỶ LỆ VỚI TRANSFORM() & ASSIGN()

- `transform()` : Để tính chỉ số theo group ở cấp dòng
- `assign()` : Để tạo hàng loạt các trường tính toán tỷ lệ một cách chuyên nghiệp

Tạo dữ liệu mẫu:

```
import pandas as pd
import numpy as np
```

```
# Tạo dữ liệu mẫu
```

```
np.random.seed(42)
```

```
df_orders = pd.DataFrame({
    'Order_ID': [f'ORD-{i:03d}' for i in range(1, 6)],
    'Region': ['North', 'South', 'North', 'South', 'North'],
    'Sales': [200, 150, 300, 450, 100]
})
```



2.2 - HƯỚNG DẪN PYTHON: TÍNH TỶ LỆ VỚI TRANSFORM() & ASSIGN()

1. Dùng transform() để tính Tổng doanh số theo Khu vực (giữ nguyên cấp độ dòng)

```
df_orders['Region_Total_Sales'] = df_orders.groupby('Region')['Sales'].transform('sum')
```

2. Dùng assign() để tạo cùng lúc nhiều trường dữ liệu tỷ lệ mới (Method Chaining)

```
grand_total_sales = df_orders['Sales'].sum()
```

```
df_orders = df_orders.assign(
```

```
    # Tính % đóng góp trên Tổng doanh số toàn cục (Grand Total)
```

```
    Pct_Grand_Total = lambda x: (x['Sales'] / grand_total_sales) * 100,
```

```
    # Tính % đóng góp trong nội bộ Khu vực (Percent within Region)
```

```
    Pct_within_Region = lambda x: (x['Sales'] / x['Region_Total_Sales']) * 100
```

```
)
```

Hiển thị kết quả

```
print(df_orders[['Order_ID', 'Region', 'Sales', 'Pct_Grand_Total', 'Pct_within_Region']])
```



2.3 - TRỰC QUAN HÓA TỶ LỆ NỘI BỘ VÀ TỶ LỆ TOÀN CỤC

Mục tiêu trực quan hóa

- ✓ **Trực quan hóa Tỷ lệ Toàn cục (Pct_Grand_Total):** Phản ánh thị phần/tỷ trọng của từng Khu vực so với toàn bộ.
- ✓ **Trực quan hóa Tỷ lệ Nội bộ (Pct_within_Region):** Phản ánh mức độ đóng góp của từng đơn vị cụ thể bên trong chính Khu vực đó (Tổng các đơn trong 1 khu vực = 100%).



2.3 - TRỰC QUAN HÓA TỶ LỆ NỘI BỘ VÀ TỶ LỆ TOÀN CỤC

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns

# Thiết lập giao diện hiển thị đồ thị cho đẹp
sns.set_theme(style="whitegrid")

# BƯỚC 1: KHOẢNG KHỞI TẠO DỮ LIỆU MẪU

np.random.seed(42) # Cố định số ngẫu nhiên để kết quả chạy luôn giống nhau

# Tạo DataFrame gồm 10 đơn hàng ngẫu nhiên thuộc 2 khu vực North và South
df_orders = pd.DataFrame(
    {
        "Order_ID": [f"ORD-{i:02d}" for i in range(1, 11)],
        "Region": np.random.choice(["North", "South"], 10),
        "Sales": np.random.randint(100, 500, 10),
    }
)
```

BƯỚC 1



HCM-UTE

2.3 - TRỰC QUAN HÓA TỶ LỆ NỘI BỘ VÀ TỶ LỆ TOÀN CỤC

BƯỚC 2: TÍNH TOÁN TỶ LỆ DỮ LIỆU (DÙNG TRANSFORM VÀ ASSIGN)

```
# 1. Dùng transform() để tính Tổng doanh số theo từng Khu vực
df_orders["Region_Total_Sales"] = df_orders.groupby("Region")["Sales"]
    .transform("sum")
```

```
# 2. Dùng assign() để tạo đồng thời các cột Tỷ lệ phần trăm mới
grand_total_sales = df_orders["Sales"].sum()
```

```
df_orders = df_orders.assign(
    # Tỷ lệ phần trăm đóng góp trên Tổng doanh số toàn cục (%)
    Pct_Grand_Total=lambda x: (x["Sales"] / grand_total_sales) * 100,
    # Tỷ lệ phần trăm đóng góp trong Nội bộ Khu vực (%)
    Pct_within_Region=lambda x: (x["Sales"] / x["Region_Total_Sales"]) * 100,
)
```

```
# In bảng dữ liệu sau khi tính toán ra màn hình
```

```
print("=" * 70)
print("BẢNG DỮ LIỆU CHI TIẾT SAU KHI TÍNH TOÁN:")
print("=" * 70)
print(
    df_orders[
        [
            "Order_ID", "Region", "Sales", "Region_Total_Sales", "Pct_Grand_Total", "Pct_within_Region",
        ]
    ].round(2)
)
print("\n")
```

BƯỚC 2

2.3 - TRỰC QUAN HÓA TỶ LỆ NỘI BỘ VÀ TỶ LỆ TOÀN CỤC

BƯỚC 3: TRỰC QUAN HÓA DỮ LIỆU TÍNH TOÁN

1. Tổng hợp dữ liệu Tỷ lệ Toàn cục cho Biểu đồ Tròn (Pie Chart)

```
region_summary = (  
    df_orders.groupby("Region")  
    .agg(  
        Total_Sales=("Sales", "sum"),  
        Pct_Grand_Total=("Pct_Grand_Total", "sum"),  
    )  
    .reset_index()  
)
```

2. Pivot dữ liệu Tỷ lệ Nội bộ cho Biểu đồ Cột Chồng (100% Stacked Bar Chart)

```
pivot_within = df_orders.pivot(  
    index="Region", columns="Order_ID", values="Pct_within_Region"  
)  
.fillna(0)
```

3. Khởi tạo khung vẽ chứa 2 biểu đồ song song (Subplots)

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(13, 5))
```

[Biểu đồ 1] Pie Chart: Tỷ lệ đóng góp TOÀN CỤC (% Grand Total)

```
ax1.pie(  
    region_summary["Pct_Grand_Total"],  
    labels=region_summary["Region"],  
    autopct="%1.1f%%",  
    colors=["#66b3ff", "#ff9999"],  
    startangle=90,  
    explode=(0.05, 0),  
)  
ax1.set_title(  
    "1. Tỷ lệ đóng góp Toàn cục theo Khu vực\n(Pct_Grand_Total)",  
    fontsize=11,  
    fontweight="bold",  
)
```

```
# [Biểu đồ 2] Stacked Bar Chart: Tỷ lệ NỘI BỘ từng Đơn hàng (% within Region)  
pivot_within.plot(kind="bar", stacked=True, ax=ax2, colormap="tab10", width=0.4)  
ax2.set_title(  
    "2. Tỷ lệ đóng góp Nội bộ của từng Đơn hàng\n(Pct_within_Region = 100%)",  
    fontsize=11,  
    fontweight="bold",  
)  
ax2.set_xlabel("Khu vực (Region)")  
ax2.set_ylabel("Tỷ lệ đóng góp (%)")  
ax2.legend(title="Order ID", bbox_to_anchor=(1.05, 1), loc="upper left")  
ax2.set_xticklabels(ax2.get_xticklabels(), rotation=0)  
plt.tight_layout()  
plt.show()
```

BƯỚC 3



2.3 - TRỰC QUAN HÓA TỶ LỆ NỘI BỘ VÀ TỶ LỆ TOÀN CỤC

BẢNG DỮ LIỆU CHI TIẾT SAU KHI TÍNH TOÁN:

	Order_ID	Region	...	Pct_Grand_Total	Pct_within_Region
0	ORD-01	North	...	9.82	16.51
1	ORD-02	South	...	13.44	33.15
2	ORD-03	North	...	5.85	9.83
3	ORD-04	North	...	14.75	24.82
4	ORD-05	North	...	6.22	10.46
5	ORD-06	South	...	14.35	35.39
6	ORD-07	North	...	7.85	13.20
7	ORD-08	North	...	7.19	12.09
8	ORD-09	North	...	7.78	13.09
9	ORD-10	South	...	12.75	31.46

[10 rows x 6 columns]

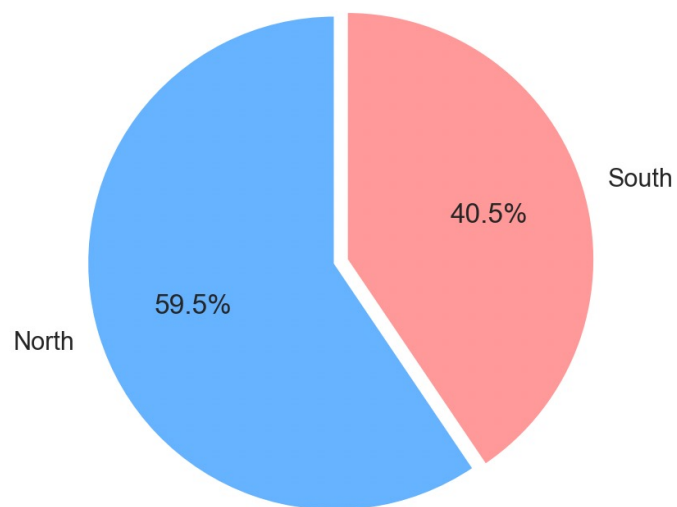
BƯỚC 3



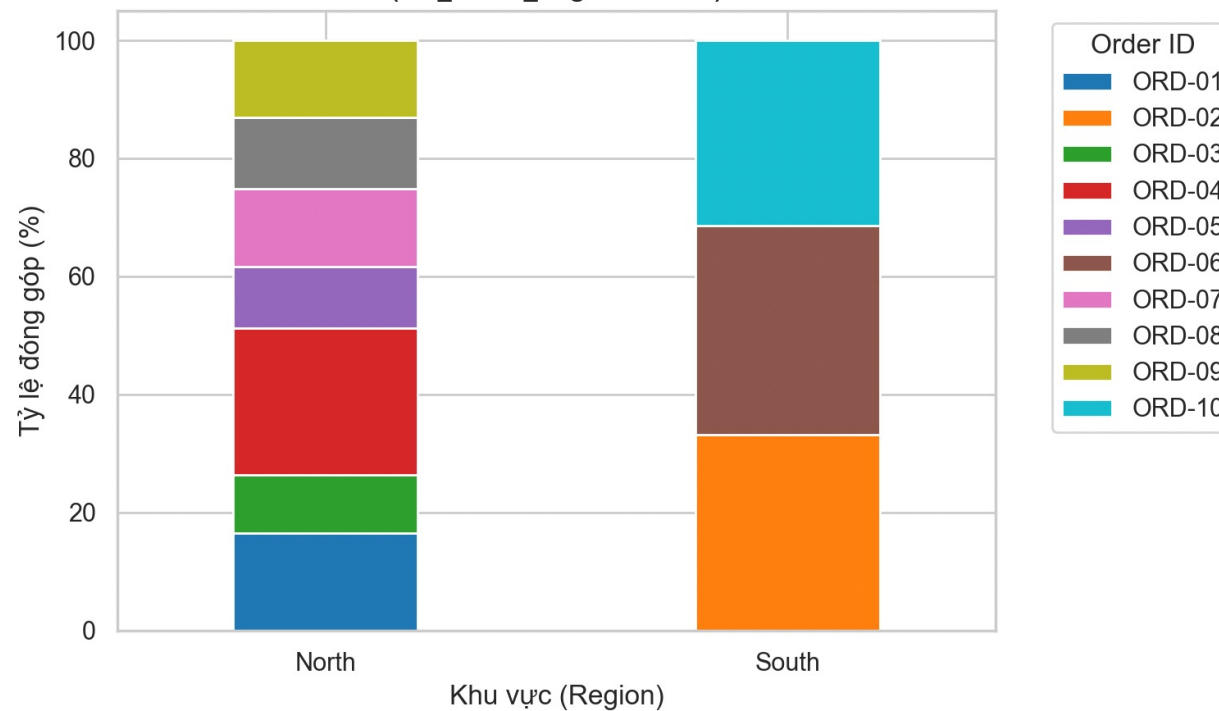
HCM-UTE

2.3 - TRỰC QUAN HÓA TỶ LỆ NỘI BỘ VÀ TỶ LỆ TOÀN CỤC

1. Tỷ lệ đóng góp Toàn cục theo Khu vực
(Pct_Grand_Total)



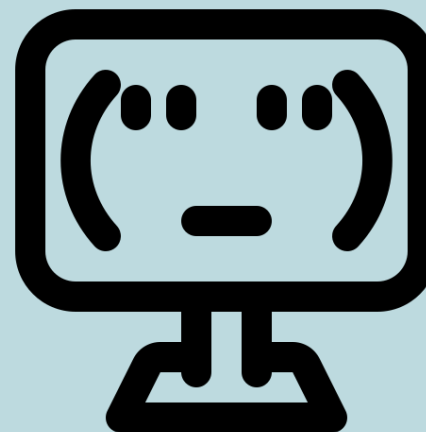
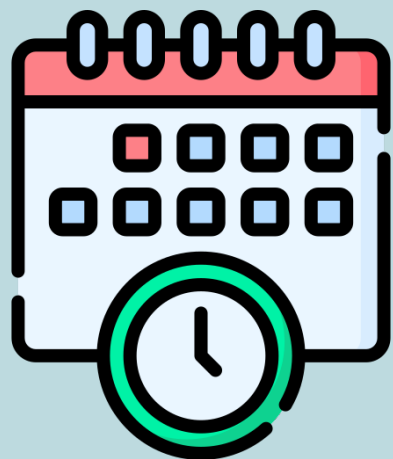
2. Tỷ lệ đóng góp Nội bộ của từng Đơn hàng
(Pct_within_Region = 100%)



BƯỚC 3



HCM-UTE



PHẦN 3: XỬ LÝ NGÀY THÁNG & CHUỖI KÝ TỰ (DATE & STRING FUNCTIONS)



3.1 - CÁC HÀM CHUỖI KÝ TỰ TRONG TRUY VẤN DỮ LIỆU

Dữ liệu danh mục thực tế thường chứa mã rác, tiền tố, hậu tố hoặc chuỗi ghép.

Bảng tóm tắt các String Functions hay dùng trong Pandas (.str)

Hàm .str	Chức năng chính	Ứng dụng điển hình
.str.split()	Tách chuỗi thành nhiều phần, có thể đưa vào nhiều cột.	Phân tách họ và tên, tách mã sản phẩm – loại sản phẩm.
.str.extract()	Trích xuất ký tự dựa trên Regular Expression (Regex).	Lấy mã số từ chuỗi văn bản, trích xuất định dạng email, số điện thoại.
.str.contains()	Kiểm tra sự tồn tại của từ khóa, trả về giá trị boolean (True/False).	Tạo cột phân loại: có chứa từ khóa “VIP” hay không.
.str.upper() / .str.lower() / .str.strip()	Chuẩn hóa dữ liệu chuỗi: viết hoa, viết thường, loại bỏ khoảng trắng dư thừa.	Làm sạch dữ liệu trước khi phân tích, đồng bộ định dạng tên, mã, nhãn.



3.2 - XỬ LÝ VÀ TÍNH TOÁN NGÀY THÁNG (DATE CALCULATIONS)

Date Functions trong Pandas (.dt)

Chuyển đổi kiểu dữ liệu:

`pd.to_datetime()` : Giúp ép kiểu chuỗi thành chuỗi thời gian chuẩn.

Trích xuất thành phần:

`.dt.year,`
`.dt.quarter,`
`.dt.month_name(),`
`.dt.day_name().`

Tính toán khoảng thời gian (Time Delta):

Khoảng cách giữa 2 mốc thời gian. (VD: Ngày giao hàng - Ngày đặt hàng = Thời gian xử lý đơn).

3.2 - XỬ LÝ VÀ TÍNH TOÁN NGÀY THÁNG (DATE CALCULATIONS)

Date Functions trong Pandas (.dt)

```
import pandas as pd
```

```
# 1. Tạo dữ liệu ban đầu dạng chuỗi văn bản (String)
```

```
df = pd.DataFrame({  
    'Order_ID': ['ORD-01', 'ORD-02'],  
    'Order_Date': ['2024-01-15', '2024-05-20'],  
    'Ship_Date': ['2024-01-18', '2024-05-26']  
})
```

	Order_ID	Order_Date	Month	Day_OfWeek	Lead_Time
0	ORD-01	2024-01-15	January	Monday	3 days
1	ORD-02	2024-05-20	May	Monday	6 days

```
# 2. Ép kiểu về datetime chuẩn với pd.to_datetime()
```

```
df['Order_Date'] = pd.to_datetime(df['Order_Date'])  
df['Ship_Date'] = pd.to_datetime(df['Ship_Date'])
```

```
# 3. Trích xuất các thành phần ngày tháng bằng .dt
```

```
df['Year'] = df['Order_Date'].dt.year  
df['Quarter'] = df['Order_Date'].dt.quarter  
df['Month'] = df['Order_Date'].dt.month_name()  
df['Day_OfWeek'] = df['Order_Date'].dt.day_name()
```

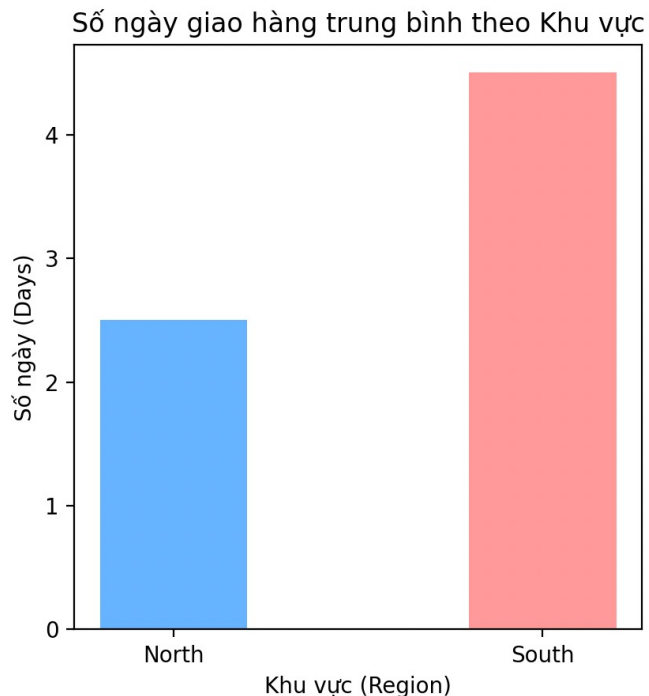
```
# Kết quả: 2024  
# Kết quả: 1 và 2  
# Kết quả: 'January', 'May'  
# Kết quả: 'Monday', 'Monday'
```

```
# 4. Tính khoảng thời gian xử lý đơn hàng (Time Delta)
```

```
df['Lead_Time'] = df['Ship_Date'] - df['Order_Date']
```

```
print(df[['Order_ID', 'Order_Date', 'Month', 'Day_OfWeek', 'Lead_Time']])
```

3.3 - THỰC HÀNH TÍNH TOÁN CHUỖI & NGÀY THÁNG



```
import pandas as pd
import matplotlib.pyplot as plt

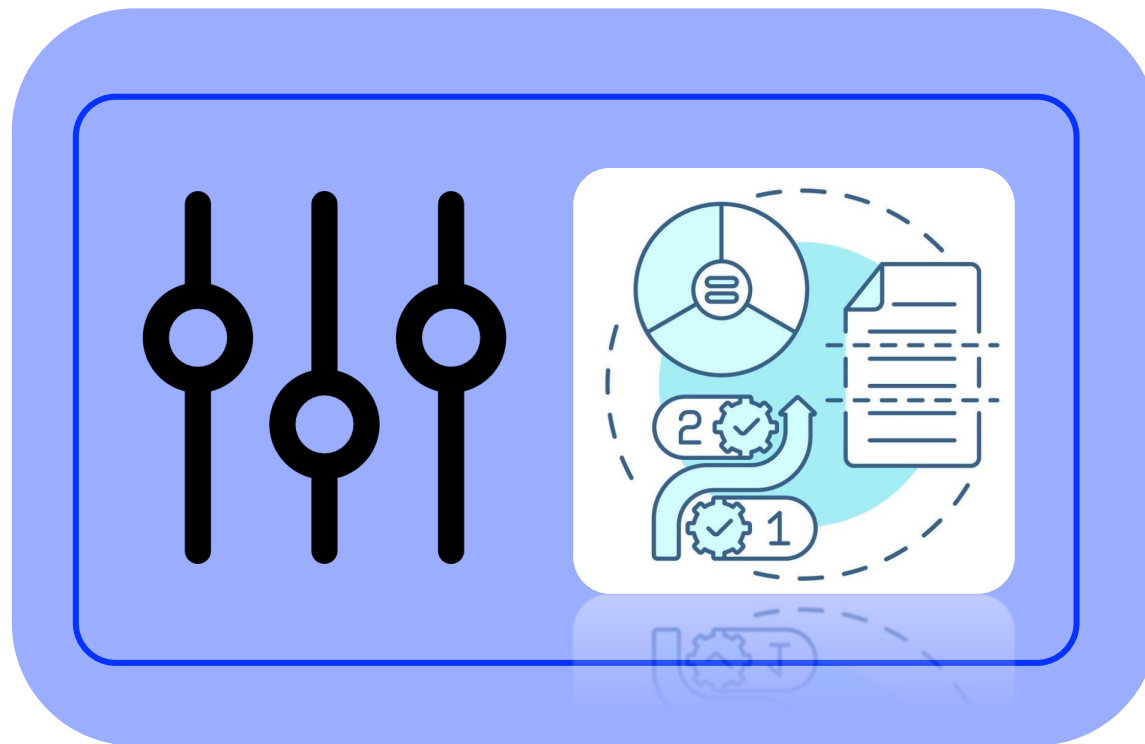
# 1. Tạo dữ liệu mẫu cực ngắn (4 đơn hàng)
df = pd.DataFrame({
    'Order_Code': ['ORD_North', 'ORD_South', 'ORD_North', 'ORD_South'],
    'Order_Date': pd.to_datetime(['2026-01-01', '2026-01-02', '2026-01-05', '2026-01-06']),
    'Ship_Date': pd.to_datetime(['2026-01-04', '2026-01-07', '2026-01-07', '2026-01-10'])
})

# 2. Xử lý Chuỗi (.str): Tách lấy tên Khu vực ('North' / 'South') từ Order_Code
df['Region'] = df['Order_Code'].str.split('_').str[1]

# 3. Xử lý Ngày tháng (.dt): Tính số ngày giao hàng = Ship_Date - Order_Date
df['Delivery_Days'] = (df['Ship_Date'] - df['Order_Date']).dt.days

# 4. Trực quan hóa: Số ngày giao hàng trung bình theo Khu vực
avg_delivery = df.groupby('Region')['Delivery_Days'].mean()

plt.bar(avg_delivery.index, avg_delivery.values, color=['#66b3ff', '#ff9999'], width=0.4)
plt.title('Số ngày giao hàng trung bình theo Khu vực')
plt.xlabel('Khu vực (Region)')
plt.ylabel('Số ngày (Days)')
plt.show()
```



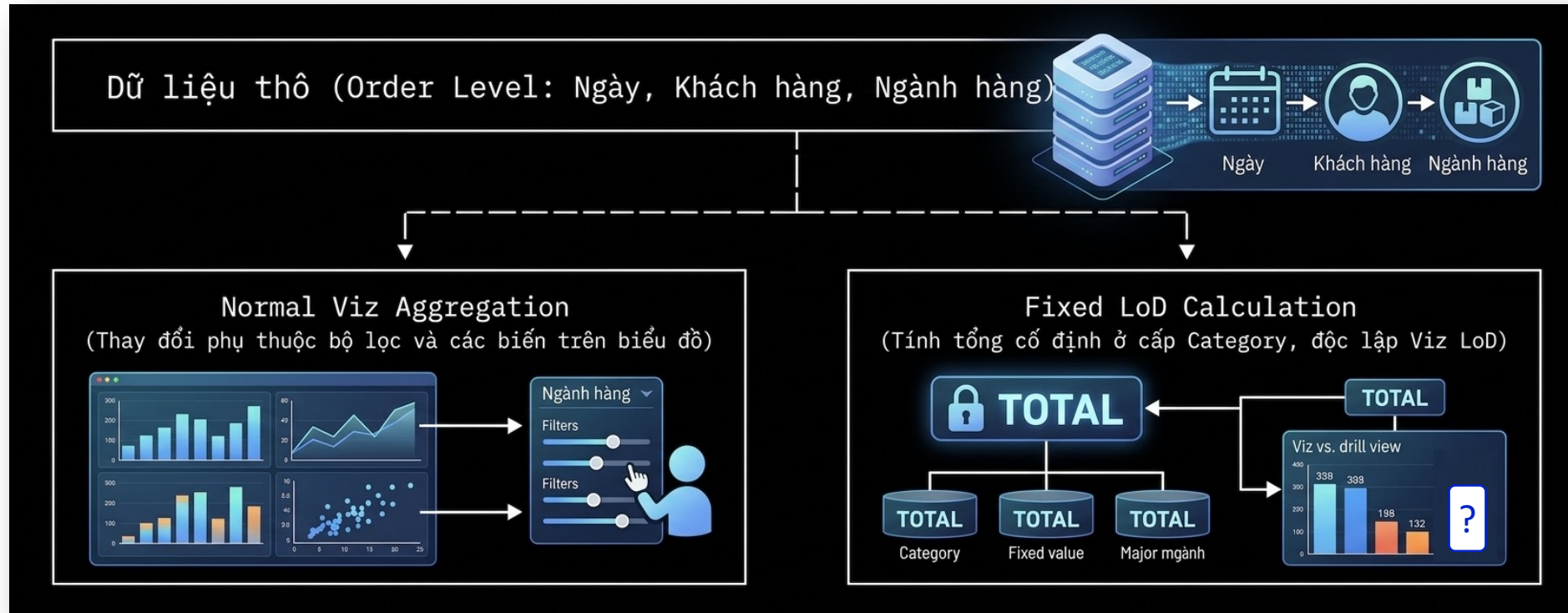
PHẦN 4: KỸ THUẬT MỨC CHI TIẾT CỐ ĐỊNH (FIXED LEVEL OF DETAIL - FIXED LOD)

4.1 - KỸ THUẬT LEVEL OF DETAIL NÂNG CAO

LoD là gì? Tại sao cần Fixed LoD?

- ☐ **Visualization LoD (Viz LoD):** Mức độ chi tiết được quyết định bởi các chiều dữ liệu (dimensions) hiển thị trên biểu đồ.
- ☐ **Fixed LoD (Fixed Level of Detail):** Cho phép tính toán các giá trị tổng hợp ở một **mức độ chi tiết cố định**, hoàn toàn **độc lập** với các trục hoặc bộ lọc đang hiển thị trên biểu đồ trực quan.
- ☐ **Ý nghĩa:** So sánh số liệu từng dòng/từng nhóm nhỏ với chỉ số Benchmark toàn cục hoặc chỉ số của nhóm cấp trên.

4.2 - CƠ CHẾ HOẠT ĐỘNG CỦA FIXED LoD



Công thức logic Fixed LoD:

Trong Python (Pandas): `df.groupby('Category')['Sales'].transform('sum')`



4.3 - HƯỚNG DẪN PYTHON: KỸ THUẬT FIXED LoD

Ý tưởng cốt lõi

Fixed LoD giúp tính giá trị tổng hợp (vd: Trung bình nhóm) và gán ngược lại từng dòng dữ liệu chi tiết bằng hàm `transform()`.

```
import pandas as pd
```

```
# 1. Tạo dữ liệu mẫu 4 đơn hàng thuộc 2 Ngành hàng
```

```
df = pd.DataFrame({  
    'Order': ['Đơn 1', 'Đơn 2', 'Đơn 3', 'Đơn 4'],  
    'Category': ['Tech', 'Tech', 'Furniture', 'Furniture'],  
    'Sales': [100, 300, 200, 400]  
})
```

```
# 2. Fixed LoD: Tính Sales trung bình cố định theo từng Ngành hàng
```

```
df['Cat_Avg_Sales'] = df.groupby('Category')['Sales'].transform('mean')
```

```
print(df)
```

	Order	Category	Sales	Cat_Avg_Sales
0	Đơn 1	Tech	100	200.0
1	Đơn 2	Tech	300	200.0
2	Đơn 3	Furniture	200	300.0
3	Đơn 4	Furniture	400	300.0

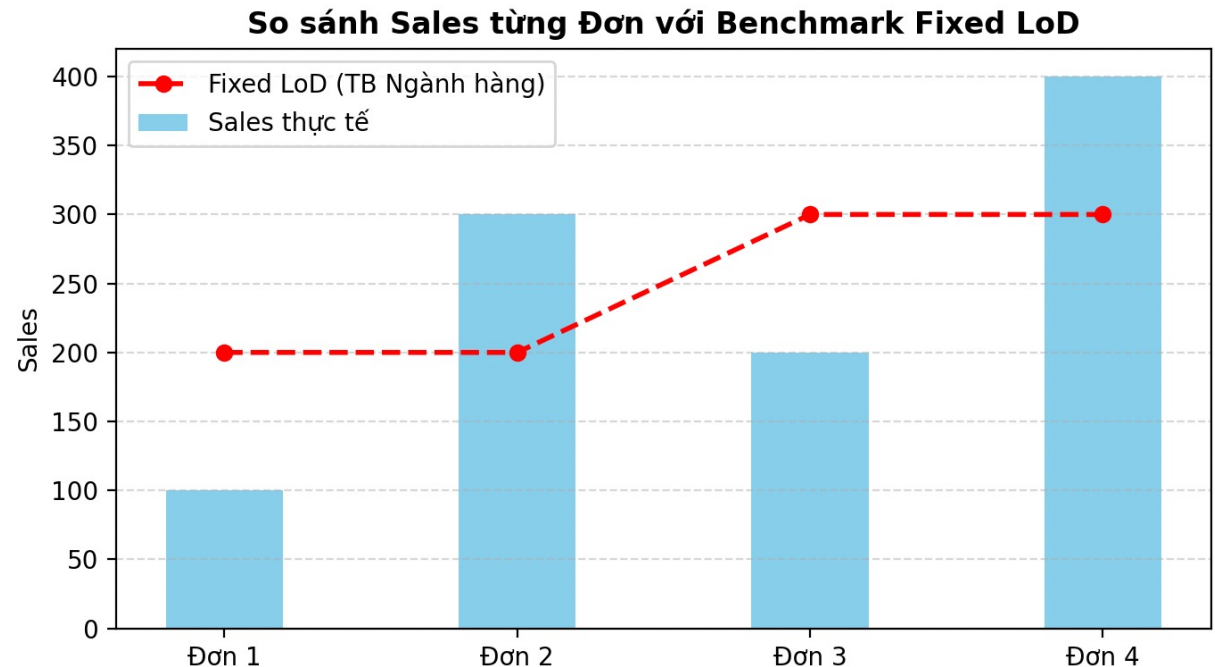
4.4 – VÍ DỤ TRỰC QUAN HÓA SO SÁNH CHI TIẾT VS. FIXED LoD BENCHMARK

Mục tiêu trực quan

Chồng **đường Benchmark Fixed LoD** (Mức trung bình nhóm) lên **biểu đồ cột Sales thực tế** của từng đơn hàng để thấy đơn nào nằm trên/dưới mức trung bình.

BẢNG DỮ LIỆU TÍNH FIXED LoD (SLIDE 14):

	Order	Category	Sales	Cat_Avg_Sales
0	Đơn 1	Tech	100	200.0
1	Đơn 2	Tech	300	200.0
2	Đơn 3	Furniture	200	300.0
3	Đơn 4	Furniture	400	300.0





4.4 – VÍ DỤ TRỰC QUAN HÓA SO SÁNH CHI TIẾT VS. FIXED LoD BENCHMARK

```
import pandas as pd
import matplotlib.pyplot as plt

# BƯỚC 1: KHOẢNG KHỞI TẠO & FIXED LoD (SLIDE 14)

df = pd.DataFrame({
    'Order': ['Đơn 1', 'Đơn 2', 'Đơn 3', 'Đơn 4'],
    'Category': ['Tech', 'Tech', 'Furniture', 'Furniture'],
    'Sales': [100, 300, 200, 400]
})

# Tính Fixed LoD (Sales trung bình theo Category)
df['Cat_Avg_Sales'] = df.groupby('Category')['Sales'].transform('mean')

print("=" * 55)
print("BẢNG DỮ LIỆU TÍNH FIXED LoD (SLIDE 14):")
print("=" * 55)
print(df)
print("\n")
```

BƯỚC 2: TRỰC QUAN HÓA BENCHMARK (SLIDE 15)

```
plt.figure(figsize=(7, 4))

# Cột doanh số chi tiết
plt.bar(df['Order'], df['Sales'], color='skyblue', width=0.4,
        label='Sales thực tế')

# Đường Benchmark Fixed LoD
plt.plot(df['Order'], df['Cat_Avg_Sales'], color='red',
        marker='o', linestyle='--', linewidth=2,
        label='Fixed LoD (TB Ngành hàng)')

plt.title('So sánh Sales từng Đơn với Benchmark Fixed LoD',
        fontsize=12, fontweight='bold')
plt.ylabel('Sales')
plt.grid(axis='y', linestyle='--', alpha=0.5)
plt.legend()
plt.tight_layout()
```



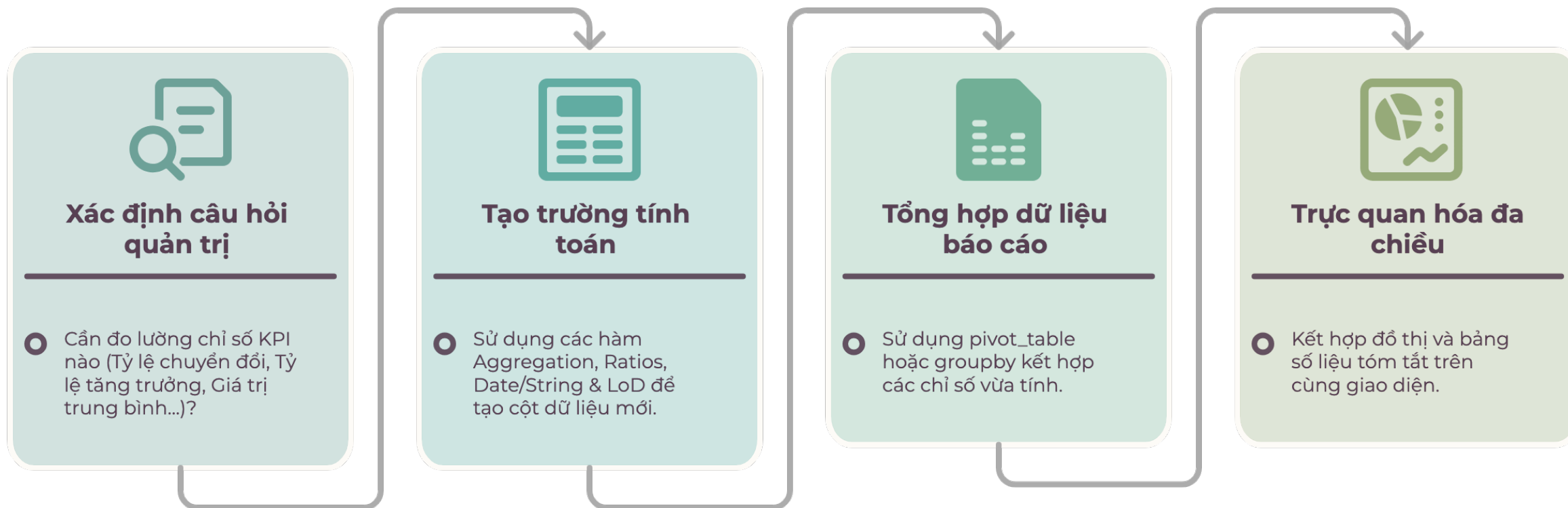
PHẦN 5: LẬP BÁO CÁO & TRỰC QUAN HÓA DỮ LIỆU TÍNH TOÁN (REPORTING & VISUAL ANALYTICS)



HCM-UTE

5.1 - XÂY DỰNG KHUNG BÁO CÁO TÍNH TOÁN

Quy trình tích hợp Calculated Fields vào Báo cáo





5.2 - HƯỚNG DẪN: TẠO BÁO CÁO ĐA BIỂU ĐỒ (SUBPLOTS & TABLES)

Mục tiêu minh họa

Trực quan hóa báo cáo tổng hợp bằng cách chia khung vẽ thành nhiều biểu đồ thành phần (`plt.subplots`) + In Bảng dữ liệu.

```
import pandas as pd
import matplotlib.pyplot as plt
```

1. KHOẢNG KHỞI TẠO DỮ LIỆU BÁO CÁO

```
report = pd.DataFrame({
    'Category': ['Tech', 'Furniture', 'Office'],
    'Sales': [5000, 3200, 1800],
    'Delivery_Days': [2.5, 4.0, 1.5]
})
```

```
print("=" * 45)
print("BẢNG BÁO CÁO TỔNG HỢP (SLIDE 17):")
print("=" * 45)
print(report)
print("\n")
```

2. VẼ BÁO CÁO ĐA BIỂU ĐỒ (SUBPLOTS)

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(9, 3.5))
```

Biểu đồ 1: Tổng Doanh thu

```
ax1.bar(report['Category'], report['Sales'], color='skyblue', width=0.4)
ax1.set_title('Tổng Doanh thu ($)', fontsize=11, fontweight='bold')
ax1.grid(axis='y', linestyle='--', alpha=0.5)
```

Biểu đồ 2: Ngày giao hàng trung bình

```
ax2.barh(report['Category'], report['Delivery_Days'], color='orange', height=0.4)
ax2.set_title('Số ngày giao trung bình', fontsize=11, fontweight='bold')
ax2.grid(axis='x', linestyle='--', alpha=0.5)
```

```
plt.tight_layout()
plt.show()
```

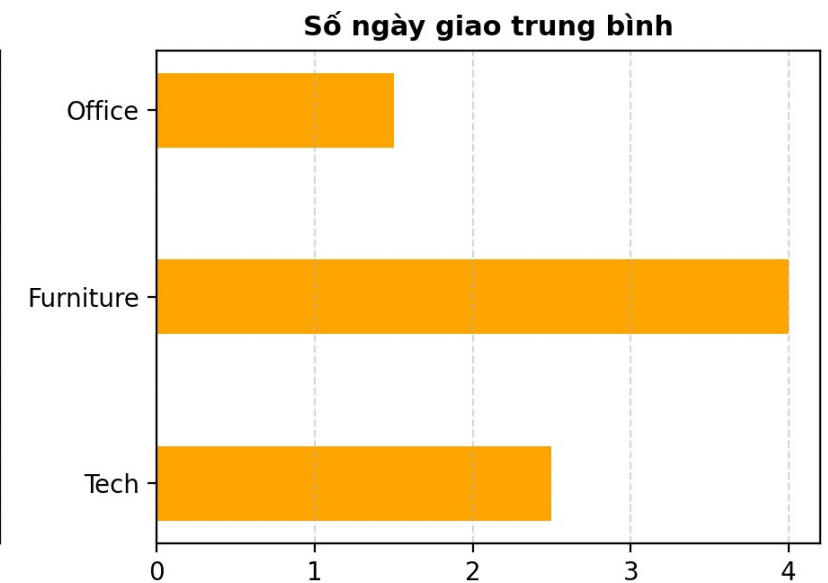
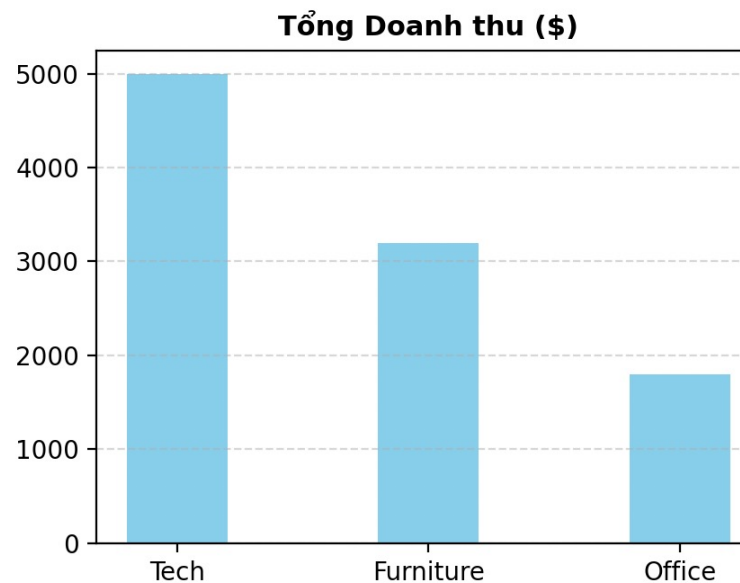
5.2 - HƯỚNG DẪN: TẠO BÁO CÁO ĐA BIỂU ĐỒ (SUBPLOTS & TABLES)

Mục tiêu minh họa

Trực quan hóa báo cáo tổng hợp bằng cách chia khung vẽ thành nhiều biểu đồ thành phần (`plt.subplots`) + In Bảng dữ liệu.

BẢNG BÁO CÁO TỔNG HỢP

	Category	Sales	Delivery_Days
0	Tech	5000	2.5
1	Furniture	3200	4.0
2	Office	1800	1.5





HCM-UTE

HO CHI MINH CITY
UNIVERSITY OF TECHNOLOGY AND ENGINEERING

01, Vo Van Ngan street, Thu Duc ward, Ho Chi Minh City, Vietnam
hcmute.edu.vn

BÀI TẬP VẬN DỤNG



Bài tập 1

Bối cảnh: Bảng quản lý vé hỗ trợ kỹ thuật (Support Tickets). Mỗi mã vé chứa thông tin mã tỉnh thành (HN, HCM, DN).

Khởi tạo dữ liệu:

```
import pandas as pd

df_tickets = pd.DataFrame({
    'Ticket_Code': ['TCK_HN_101', 'TCK_HCM_102', 'TCK_HN_103', 'TCK_DN_104', 'TCK_HCM_105'],
    'Customer_ID': ['CUST_A', 'CUST_B', 'CUST_A', 'CUST_C', 'CUST_B']
})
```

Yêu cầu:

1. Trích xuất tên Thành phố (HN, HCM, DN) từ Ticket_Code sang cột City.
2. Đếm số lượng khách hàng duy nhất (nunique) có gửi yêu cầu theo từng Thành phố.
3. Lưu bài: C5_hovaten_MSSV_baitap1.ipynb



HCM-UTE

Bài tập 2

Bối cảnh: Bảng dữ liệu đặt phòng khách sạn (`Hotel Bookings`) lưu mốc thời gian Check-in, Check-out và tổng tiền.

Khởi tạo dữ liệu:

```
import pandas as pd

df_hotels = pd.DataFrame({
    'Booking_ID': ['BK-01', 'BK-02', 'BK-03'],
    'Check_In': pd.to_datetime(['2026-03-01', '2026-03-02', '2026-03-05']),
    'Check_Out': pd.to_datetime(['2026-03-04', '2026-03-07', '2026-03-06']),
    'Total_Fee': [300, 1000, 200]
})
```

Yêu cầu:

1. Tính số ngày lưu trú: `Stay_Days`.
2. Tạo cột `Daily_Rate` thể hiện chi phí bình quân mỗi ngày.
3. Lưu bài: Lưu bài: `C5_hovaten_MSSV_baitap2.ipynb`



Bài tập 3

Bối cảnh: Bảng ghi nhận doanh số bán hàng của nhân viên theo từng Phòng ban.

Khởi tạo dữ liệu:

```
import pandas as pd

df_staff = pd.DataFrame({
    'Emp_Name': ['An', 'Bình', 'Cường', 'Dung', 'Giang'],
    'Dept': ['Sales', 'Sales', 'IT', 'IT', 'Sales'],
    'Sales': [120, 200, 150, 180, 90]
})
```

Yêu cầu:

1. Dùng kỹ thuật **Fixed LoD** tạo cột `Dept_Max_Sales` thể hiện mức doanh số cao nhất của phòng ban mà nhân viên đó thuộc về.
2. Tính độ lệch doanh số cá nhân so với đỉnh phòng ban: `Diff_To_Max`
3. Lưu bài: Lưu bài: `C5_hovaten_MSSV_baitap3.ipynb`



HCM-UTE

Mini-Lab: Phân tích hiệu suất giao hàng ứng dụng Food Delivery

Khởi tạo dữ liệu đầu bài:

```
import pandas as pd
import numpy as np

# Dữ liệu 6 đơn hàng ứng dụng giao đồ ăn
df_food = pd.DataFrame({
    'Order_Code': ['FD_HN_01', 'FD_HCM_02', 'FD_HN_03', 'FD_DN_04',
                  'FD_HCM_05', 'FD_HN_06'],
    'Cust_ID': ['C01', 'C02', 'C01', 'C03', 'C04', 'C02'],
    'Order_Time': pd.to_datetime(['2026-04-01 10:00', '2026-04-01 11:15',
                                  '2026-04-01 11:30',
                                  '2026-04-01 12:00', '2026-04-01 12:15',
                                  '2026-04-01 12:45']),
    'Deliver_Time': pd.to_datetime(['2026-04-01 10:25', '2026-04-01 11:50',
                                    '2026-04-01 12:10',
                                    '2026-04-01 12:40', '2026-04-01 12:35',
                                    '2026-04-01 13:15']),
    'Amount': [150, 320, 210, 90, 450, 180]
})
```

Mini-Lab: Phân tích hiệu suất giao hàng ứng dụng Food Delivery

Yêu cầu thực hiện:

Bước 1 (Xử lý Chuỗi & Thời gian):

- Tách tên Thành phố (HN, HCM, DN) từ `Order_Code` thành cột `City`.
- Tính thời gian giao hàng theo phút: `Delivery_Mins = (Deliver_Time - Order_Time).dt.total_seconds() / 60`.

Bước 2 (Aggregation & Count Distinct):

- Gom nhóm theo `City`, tính: Tổng doanh thu (`Amount`), Thời gian giao trung bình (`Delivery_Mins`) và Số lượng khách duy nhất (`Cust_ID`).

Bước 3 (Fixed LoD & Ratio):

- Dùng Fixed LoD tính `Fixed_City_Sales` (Tổng doanh thu cố định của từng Thành phố).
- Tính tỷ lệ % đóng góp doanh thu của mỗi đơn hàng trong Thành phố đó (`Amount / Fixed_City_Sales * 100`).

Bước 4 (Trực quan hóa đa chiều - Subplots):

- Ô 1: Biểu đồ cột thể hiện Tổng doanh thu theo từng Thành phố (`City`).
- Ô 2: Biểu đồ cột ngang thể hiện Thời gian giao hàng trung bình (phút) theo Thành phố (`City`).

Lưu bài: [\[C5_hovaten_MSSV_minilab.ipynb\]](#)



THANK YOU !